

Aula 1 - Introdução à Qualidade de Software

Duração: 2 horas

Objetivos:

- Compreender os conceitos fundamentais de Qualidade de Software.
- Relacionar qualidade com boas práticas de desenvolvimento.
- Aplicar um exemplo prático em Java para demonstrar Clean Code.

Parte Teórica (1h)

1. O que é Qualidade de Software?

- **Definição:** Atender aos requisitos do usuário, ser mantível, eficiente e livre de defeitos.
- **Padrões e Modelos:**
 - ISO 25010 (Qualidade de Produto de Software)
 - CMMI (Capability Maturity Model Integration)
 - MPS.BR (Melhoria de Processo do Software Brasileiro)

2. Por que Qualidade é Importante?

- Reduz retrabalho e custos.
- Facilita a manutenção e escalabilidade.
- Melhora a experiência do usuário.

3. Pilares da Qualidade de Software

1. **Funcionalidade** (Atende aos requisitos?)
2. **Confiabilidade** (Falha com frequência?)
3. **Usabilidade** (Fácil de usar?)
4. **Eficiência** (Desempenho adequado?)
5. **Manutenibilidade** (Fácil de modificar?)
6. **Portabilidade** (Roda em diferentes ambientes?)

4. Clean Code e Boas Práticas

- Nomes significativos para variáveis e métodos.

- Funções pequenas e com única responsabilidade.
- Evitar código duplicado (DRY - Don't Repeat Yourself).

Parte Prática (1h) - Aplicando Clean Code em Java

Atividade: Refatoração de Código

Cenário: Um sistema de cálculo de salário com problemas de qualidade.

Código Inicial (Ruim)

```
java

public class CalculoSalario {
    public static void main(String[] args) {
        double s = 5000;
        double d = calcular(s);
        System.out.println("Salário líquido: " + d);
    }

    public static double calcular(double sl) {
        double r = 0;
        if (sl < 1000) r = sl * 0.05;
        else if (sl < 2000) r = sl * 0.1;
        else if (sl < 4000) r = sl * 0.15;
        else r = sl * 0.2;
        return sl - r;
    }
}
```

Problemas identificados:

- Nomes de variáveis obscuros (s , d , sl , r).
- Método pouco legível.
- Falta de tratamento para valores negativos.

Refatoração (Código Limpo)

```
java

public class SalaryCalculator {
    public static void main(String[] args) {
```

```

    double grossSalary = 5000.0;
    double netSalary = calculateNetSalary(grossSalary);
    System.out.println("Net Salary: " + netSalary);
}

public static double calculateNetSalary(double grossSalary) {
    if (grossSalary < 0) {
        throw new IllegalArgumentException("Salary cannot be negative.");
    }

    double taxRate = getTaxRate(grossSalary);
    double taxAmount = grossSalary * taxRate;
    return grossSalary - taxAmount;
}

private static double getTaxRate(double salary) {
    if (salary < 1000) return 0.05;
    else if (salary < 2000) return 0.10;
    else if (salary < 4000) return 0.15;
    else return 0.20;
}
}

```

Melhorias aplicadas:

- ✓ **Nomes descritivos** (grossSalary , netSalary , taxRate).
- ✓ **Métodos pequenos e especializados** (getTaxRate separado da lógica principal).
- ✓ **Validação de entrada** (evita salários negativos).
- ✓ **Legibilidade e manutenibilidade**.

Exercício Prático para os Alunos (20 min)

Problema: Refatore o código abaixo aplicando Clean Code.

java

```

public class OrderProcessor {
    public double calcTotal(double[] prices, int qty[]) {
        double t = 0;
        for (int i = 0; i < prices.length; i++) {
            t += prices[i] * qty[i];
        }
        if (t > 1000) {
            t = t * 0.9;
        }
        return t;
    }
}

```

```
}  
}
```

Dicas:

- Melhore os nomes das variáveis e métodos.
- Extraia lógica de desconto para um método separado.
- Adicione validações (ex.: arrays não podem ser nulos, tamanhos devem ser iguais).

Encerramento

- **Discussão:** Qual impacto a refatoração tem na qualidade do software?
- **Próxima Aula:** Metodologias Ágeis e Qualidade (Scrum, CI/CD).

Tarefa Opcional:

- Pesquisar sobre ferramentas de análise estática (SonarQube, Checkstyle).

Recursos:

- [Documentação Clean Code \(Robert C. Martin\)](#)
- [ISO 25010 - Qualidade de Software](#)

Espero que essa aula seja produtiva! Se precisar de ajustes, é só avisar. 😊🚀